Artifact Narrative – Enhanced Dashboard & MongoDB Integration

Artifact Description

The artifact for this enhancement is the Grazioso Salvare Dashboard and its MongoDB-backed data pipeline. The original version included basic CRUD operations, simple rescue filters, and a dashboard that loaded the entire AAC dataset without validation, indexing, or analytics. For this enhancement, I significantly expanded the database functionality, improved data reliability, and redesigned the analytics section to provide meaningful insights.

Justification for Inclusion

I selected this artifact because it demonstrates my ability to design and integrate a real-world database solution. Enhancing the database layer allowed me to showcase skills in schema validation, indexing, aggregation pipelines, secure authentication, and data cleaning. These improvements directly align with the Databas

es outcome and demonstrate my readiness to work with production-grade data systems.

Key Enhancements Completed

1. Secure Credential Management I removed hardcoded MongoDB credentials and replaced them with a dedicated login_page.py module and environment-based configuration.

Before: Credentials stored directly in the dashboard file.

After: Credentials retrieved securely using get_credentials(), improving maintainability and security.

2. Schema Validation & Indexing I added MongoDB JSON schema validation to enforce required fields and prevent malformed data. I also created indexes on frequently queried fields such as breed, animal_type, and outcome_type. This improves query performance and ensures consistent data integrity.

3. Aggregation Pipelines for Analytics I redesigned the analytics section using MongoDB aggregation pipelines to compute:

- Breed distribution
- Outcome distribution
- Age statistics (min, max, average)

- Rescue availability grouped by type and outcome

These pipelines transform raw records into actionable insights and demonstrate advanced database querying skills.

4. Robust Data Cleaning The AAC dataset contains invalid numeric values such as "not_a_number". I implemented numeric coercion using pd.to_numeric(errors="coerce") to prevent crashes and ensure analytics always render correctly.

5. UI/UX Improvements for Data Visualization I replaced the original heatmap and line chart with more appropriate visualizations:

- Grouped bar chart (breed vs outcome)
- Histogram (age distribution)
- KPI tiles (total animals, average age, most common breed, most common outcome)

These changes make the dashboard more intuitive and data-driven.

Testing Evidence

Testing confirmed that the database enhancements worked as intended:

- Aggregation pipelines returned correct counts and sorted results. Example: Outcome distribution correctly showed “Adoption” as the most common outcome in the filtered dataset.
- Age statistics pipeline produced accurate min/max/average values after numeric coercion.
- Indexing improved query responsiveness, especially for breed-based rescue filters.
- Schema validation correctly warned when fields such as animal_type or outcome_type were missing.
- Invalid numeric values were safely converted to NaN, preventing crashes in the histogram and KPI tiles.

These results demonstrate measurable improvements in reliability, performance, and data quality.

Reflection on the Enhancement Process

Enhancing this artifact taught me how to design resilient, secure, and scalable database systems. One challenge was handling inconsistent AAC data; numeric coercion and schema validation helped ensure that analytics remained stable. Another challenge was integrating aggregation results into the dashboard without breaking existing callbacks; careful testing and incremental updates resolved this.